Rivanna Runbook — Faithful 537 Sweep

These files run the faithful `537_Sports_Simulation` parameter sweep as Slurm job arrays, without adding any new mechanisms.

Files

- `faithful_537_sweep.py` — sweep grid and scenario runner.
- `faithful_537_sweep_rivanna_worker.py` — Rivanna worker (`stage1-shard`, `merge-stage1`, `stage2-shard`, `merge`).
- `rivanna_stage1_faithful_537.slurm` — **Stage 1 array** (default 64 tasks): shard CSVs under `stage1_shards/`.
- `rivanna_merge_stage1_faithful_537.slurm` — `merge-stage1`: concatenate shards → `stage1_results.csv`.
- `rivanna_stage2_array_faithful_537.slurm` — Stage 2 verification array (default 64 tasks).
- `rivanna_merge_faithful_537.slurm` — merge Stage 2 shards, grouped candidates, plots.

Single-process Stage 1 (no Slurm array):
`python faithful_537_sweep_rivanna_worker.py stage1 --reset` writes `stage1_results.csv` directly (debug / small tests).

Rivanna / Slurm Conventions

These scripts mirror the repo-level `pipe_job.slurm` conventions:

- `#SBATCH --account=sds_ag`
- `#SBATCH --mail-user=dzk3ja@virginia.edu`
- `#SBATCH --constraint=rivanna`
- `module load miniforge`
- `ENV_NAME="${ENV_NAME:-sports_net}"`
- line-buffered stdout/stderr via `stdbuf` when available

The scripts first try:

```
$HOME/.conda/envs/sports_net/bin/python
```

and fall back to `python` on PATH.

Submit From Repo Root

One file (like `pipe_job.slurm`)

From the **repository root** (same directory as `pipe_job.slurm`):

```
sbatch sim_job.slurm
```

That driver job submits **Stage 1 array → Merge Stage 1 → Stage 2 array → final merge** and uses `sbatch --wait` on each step so one `sbatch` submission runs the full pipeline. Override shard counts if needed:

```
N_STAGE1_SHARDS=64 N_SHARDS=64 sbatch sim_job.slurm
```

The driver needs a long enough `#SBATCH --time` (default **12 hours**) because it stays active until all child jobs finish.

Manual dependency chain

On Rivanna, `cd` to the repository root (the directory containing `sports/`), then use a dependency chain so **Merge Stage 1** runs only after **all** Stage 1 array tasks finish:

```
j1=$(sbatch --parsable sports/outputs/simulation_sweeps/rivanna_stage1_faithful_537.slurm)
j1m=$(sbatch --parsable --dependency=afterok:$j1 sports/outputs/simulation_sweeps/rivanna_merge_stage1_faithful_537.slurm)
j2=$(sbatch --parsable --dependency=afterok:$j1m sports/outputs/simulation_sweeps/rivanna_stage2_array_faithful_537.slurm)
sbatch --dependency=afterok:$j2 sports/outputs/simulation_sweeps/rivanna_merge_faithful_537.slurm
```

(`afterok` on the Stage 1 job id waits for the **entire** array to complete successfully.)

Outputs

All outputs land under:

```
sports/outputs/simulation_sweeps/rivanna_faithful_537/
```

Important outputs:

- `stage1_shards/stage1_shard_*.csv`
- `stage1_results.csv` (after `merge-stage1`)
- `stage2_shards/stage2_shard_*.csv`
- `stage2_results_merged.csv`
- `grouped_candidates.csv`
- `candidate_plots/`
- `README.md`

Shards

Stage 1 (default 64 tasks):

```
#SBATCH --array=0-63
N_STAGE1_SHARDS="${N_STAGE1_SHARDS:-64}"
```

If you change the array range, set `N_STAGE1_SHARDS` to match when submitting **Merge Stage 1**, e.g.:

```
#SBATCH --array=0-127
N_STAGE1_SHARDS=128 sbatch sports/outputs/simulation_sweeps/rivanna_merge_stage1_faithful_537.slurm
```

Stage 2 (default 64 tasks):

```
#SBATCH --array=0-63
N_SHARDS="${N_SHARDS:-64}"
```

If you change the Stage 2 array range, set `N_SHARDS` to match for **both** the Stage 2 array and the **final** merge script. Example for 128 shards:

```
#SBATCH --array=0-127
N_SHARDS=128 sbatch sports/outputs/simulation_sweeps/rivanna_stage2_array_faithful_537.slurm
N_SHARDS=128 sbatch sports/outputs/simulation_sweeps/rivanna_merge_faithful_537.slurm
```